IY3660 – Applications of Cryptography

Week 7 Lecture
Authenticated Encryption

Elizabeth Quaglia
Information Security Group

Quiz time!

In today's lecture

- (Recap) formal security notions for block ciphers
- Definition of Authenticated Encryption (AE)
- Generic construction of AE
- Dedicated constructions of AE
- The CAESAR competition

Formalising symmetric encryption

Formal definition of symmetric encryption

- A symmetric encryption scheme consists of a triple of algorithms:

  (**KGen**, **Enc**, **Dec**)

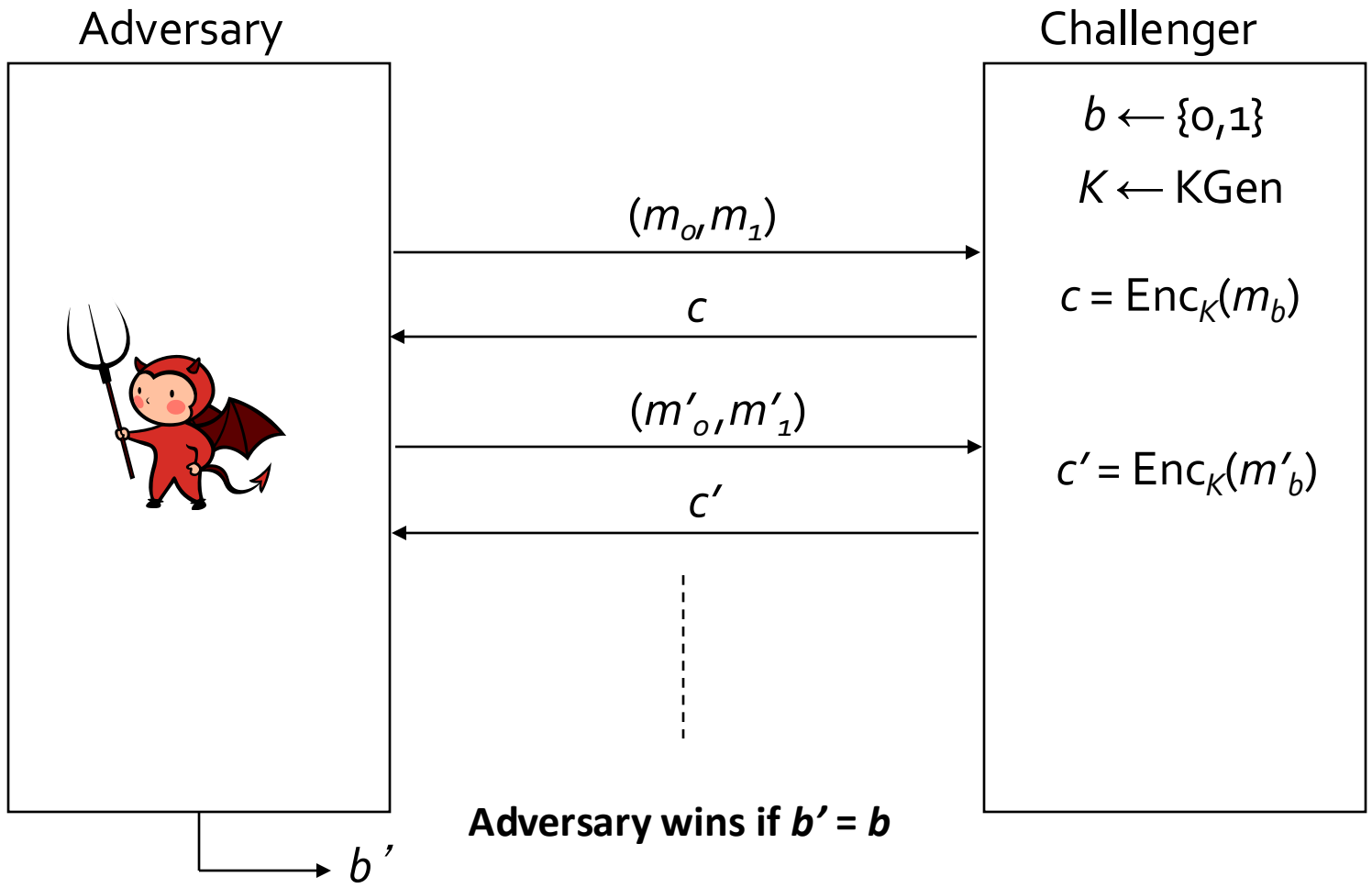
  - **KGen**: key generation, selects a key K uniformly at random from $\{0,1\}^k$
  - **Enc**: encryption, takes as input key K, plaintext $m \in \{0, 1\}^*$ and produces output $c \in \{0, 1\}^*$
  - **Dec**: decryption, takes as input key K, ciphertext $c \in \{0, 1\}^*$ and produces output $m \in \{0, 1\}^*$ or an error message, denoted as $\perp$.

  Correctness: we require that for all keys K, and for all plaintexts m, $\mathbf{Dec}_K(\mathbf{Enc}_K(m)) = m$

Notes:

- **Enc** may be *randomised* or *stateful* (cf. CBC mode, CTR mode)
- There will be a maximum plaintext length that can be encrypted

IND-CPA: indistinguishability under chosen-plaintext attack



Initialise: the oracle (*challenger*) draws at random a secret encryption key K and a bit b= 0 or 1

If this Advantage is small, then we say that Enc is **IND-CPA secure**

$$\mathrm{Adv}_E^{IND\text{-}CPA}(A) := 2.\Pr[b'=b] - 1$$

IND-CPA security

- The adversary's advantage in the IND-CPA security game is:

$$\text{Adv} = 2\Pr[b=b'] - 1$$

- Formulated as such because adversary can always just guess with probability 1/2 of being correct!

- A symmetric encryption scheme SE is said to be IND-CPA secure if the advantage is "small" for *any* adversary using "reasonable" resources
  - Concepts of "small" and "reasonable" can be formalised but are beyond the scope of this course
  - It can be proved that schemes like CBC-mode and CTR-mode meet this security definition if used properly and if they are built using a good block cipher

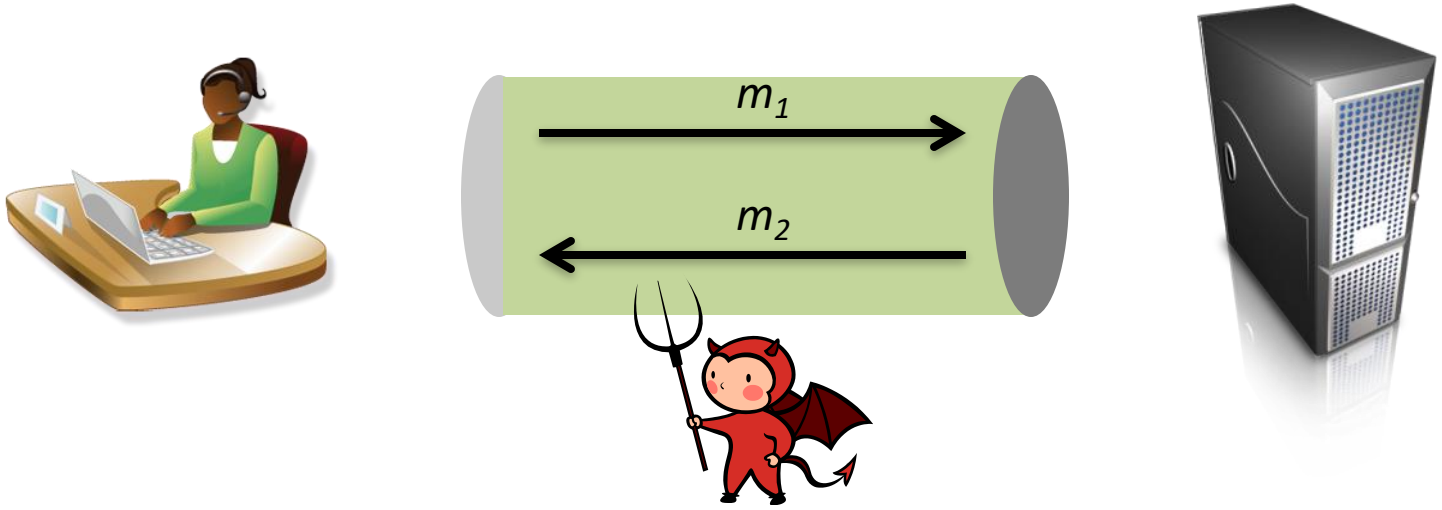
Motivating stronger security

- In the IND-CPA game, the adversary has access only to encryption
- For some modes we have seen how an active adversary can manipulate ciphertexts
  - Can learn information from how these are decrypted
- For CTR mode, bit flipping in plaintext can be done by bit flipping in the ciphertext
  - Modify c to $c \oplus \Delta$ to change the underlying plaintext from p to $p \oplus \Delta$
- CBC mode: see padding oracle attacks
- It is possible to create completely new ciphertexts from scratch
  - A random string of bits of the right length is a valid ciphertext for some plaintext for both CBC and CTR modes!

Motivating stronger security

- These kinds of attack do not break **IND-CPA** security
- However, they are clearly undesirable if we want to build secure channels
  - A modified plaintext may result in wrong message being delivered to an application, or unpredictable behaviour at the receiving application
- We really want some kind of **non-malleable** encryption
  - Guaranteeing integrity as well as confidentiality
- We can consider two basic security notions:
  - integrity of plaintexts
  - integrity of ciphertexts
- This motivates: **Authenticated Encryption**

Authenticated Encryption (AE)



Security goals:

- Confidentiality and integrity of messages exchanged between Alice and Bob

Adversarial capabilities:

- Adversary can arbitrarily delete, reorder, modify (etc) bits on the wire
- Adversary can mount chosen plaintext and chosen ciphertexts attacks – formalised via encryption and decryption oracles

Tools we now have:

- Encryption (e.g., CBC mode, CTR mode) and MAC algorithms (e.g., HMAC)

Authenticated Encryption – informal definition

A symmetric encryption scheme is said to offer **Authenticated Encryption security** if:

- A chosen plaintext attacker can learn nothing about plaintexts from ciphertexts (except their lengths)

AND

- An attacker with access to an encryption oracle cannot forge any new ciphertexts

Notes:

- How do we formalise "cannot forge any new ciphertexts"?
- Why is that property important anyway?

Integrity of ciphertexts – INT-CTXT

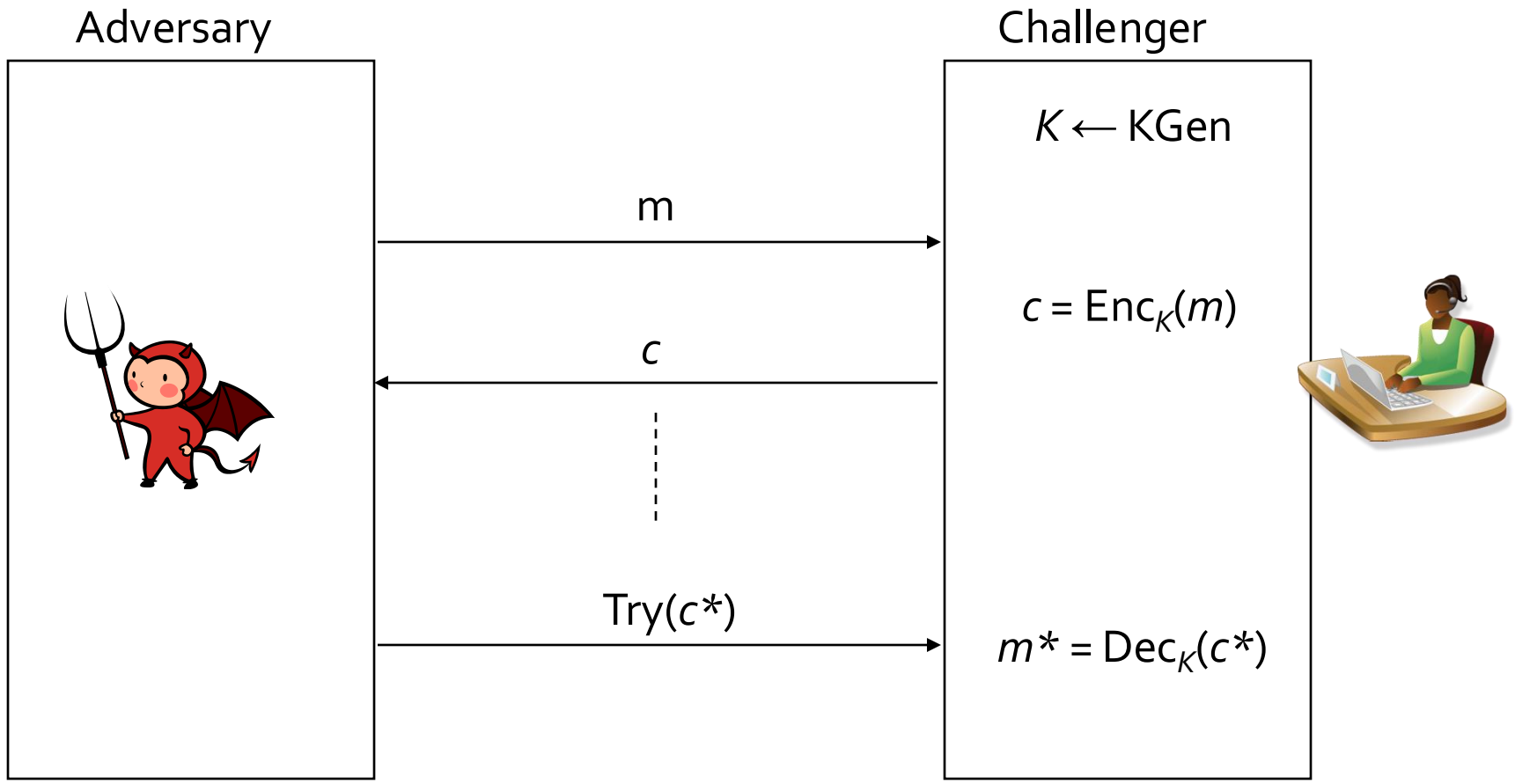
- Adversary's goal is to get the receiver to accept a "non-authentic" ciphertext
- Attacker has repeated access to an encryption oracle and a "Try" oracle
  - The Encryption oracle takes any m as input, and outputs $\mathrm{Enc}_K(m)$
  - The Try oracle takes any c* as input (and has no output)

- Adversary's task is to submit a ciphertext c^* to its Try oracle such that:
  1. c^* is distinct from all the ciphertexts c that were output by the encryption oracle; and
  2. $\mathrm{Dec}_K(c^*)$ decrypts to message $m^* \neq \perp$ (in other words, it is a valid ciphertext)

- Hence adversary wins if it can create a "ciphertext forgery"
  - A **new** ciphertext that it did not get from its encryption oracle

- m^* need not be different from all the m queried to the encryption oracle, only c^* must be different

INT-CTXT: Integrity of Ciphertexts security



If the success probability of any adversary using reasonable resources is small, then we say that Enc is **INT-CTXT secure**

Adversary wins if c^* is "new" and $m^* \neq \perp$

AE Security

Recall that a symmetric encryption scheme is said to offer Authenticated Encryption security if:

A chosen plaintext attacker can learn nothing about plaintexts from ciphertexts (except their lengths)

AND

An attacker with access to an encryption oracle cannot forge any new ciphertexts

- More formally, we can now say that:

AE = IND-CPA + INT-CTXT

Generic constructions of AE

Generic composition for AE

- Idea: we wish to construct a secure authenticated encryption scheme based on existing symmetric encryption schemes and message authentication schemes

Recall the security notions for these primitives:

- IND-CPA security: indistinguishability under chosen plaintext attacks
- UF-CMA security: (existential) unforgeability under chosen message attack
- SUF-CMA security: a stronger notion of security for message authentication

Generic composition for AE

- Assume we have IND-CPA secure encryption schemes
  - e.g., CBC mode, CTR mode
- Assume we have SUF-CMA secure MAC schemes
  - e.g., HMAC
- Can we combine these to obtain AE security for symmetric encryption?
- Generic options:
  - E&M: Encrypt and MAC
  - MtE: MAC then Encrypt
  - EtM: Encrypt then MAC
- NB: in what follows, *KM* denotes a MAC key, and *KE* an encryption key

Generic composition for AE

Encrypt-and-MAC (E&M)

- compute $c' \leftarrow \mathrm{Enc}_{KE}(m)$ and $\tau \leftarrow \mathrm{Tag}_{KM}(m)$ and output $c = (c', \tau)$
- used in SSH

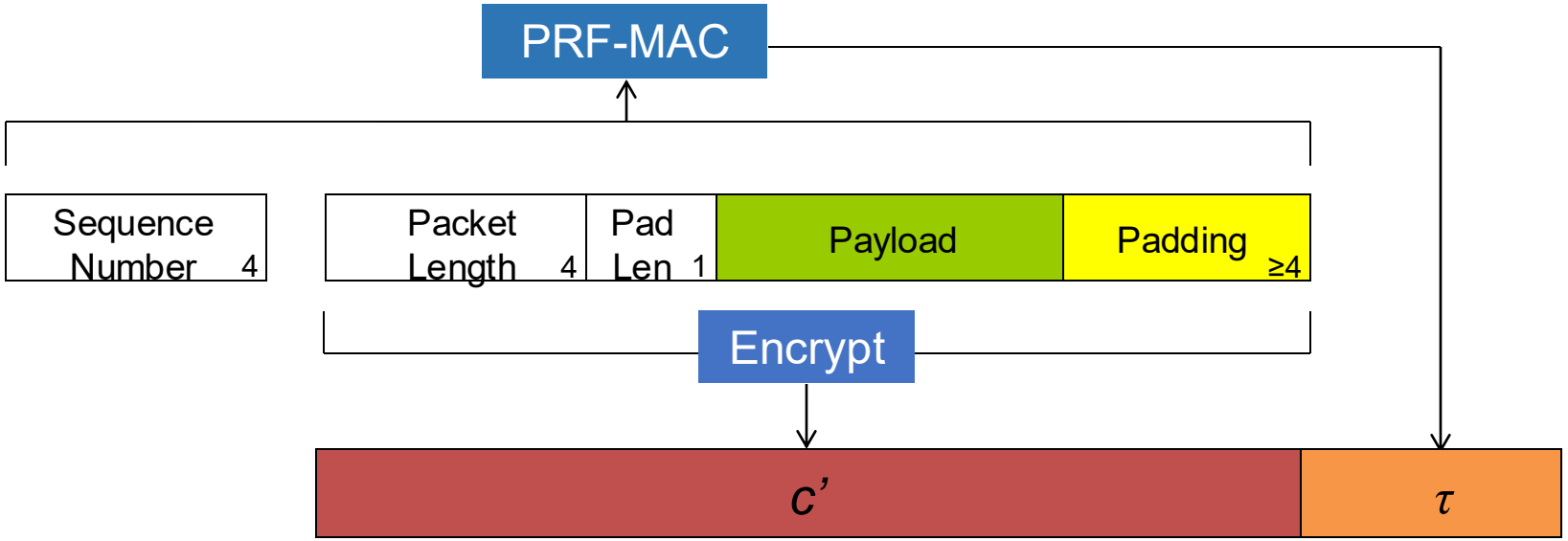
MAC-then-Encrypt (MtE)

- compute $\tau \leftarrow \mathrm{Tag}_{KM}(m)$ and output $c = \mathrm{Enc}_{KE}(m \mid\mid \tau)$
- used in TLS

Encrypt-then-MAC (EtM)

- compute $c' \leftarrow \mathrm{Enc}_{KE}(m)$ and $\tau \leftarrow \mathrm{Tag}_{KM}(c')$ and output $c = (c', \tau)$
- used in IPsec

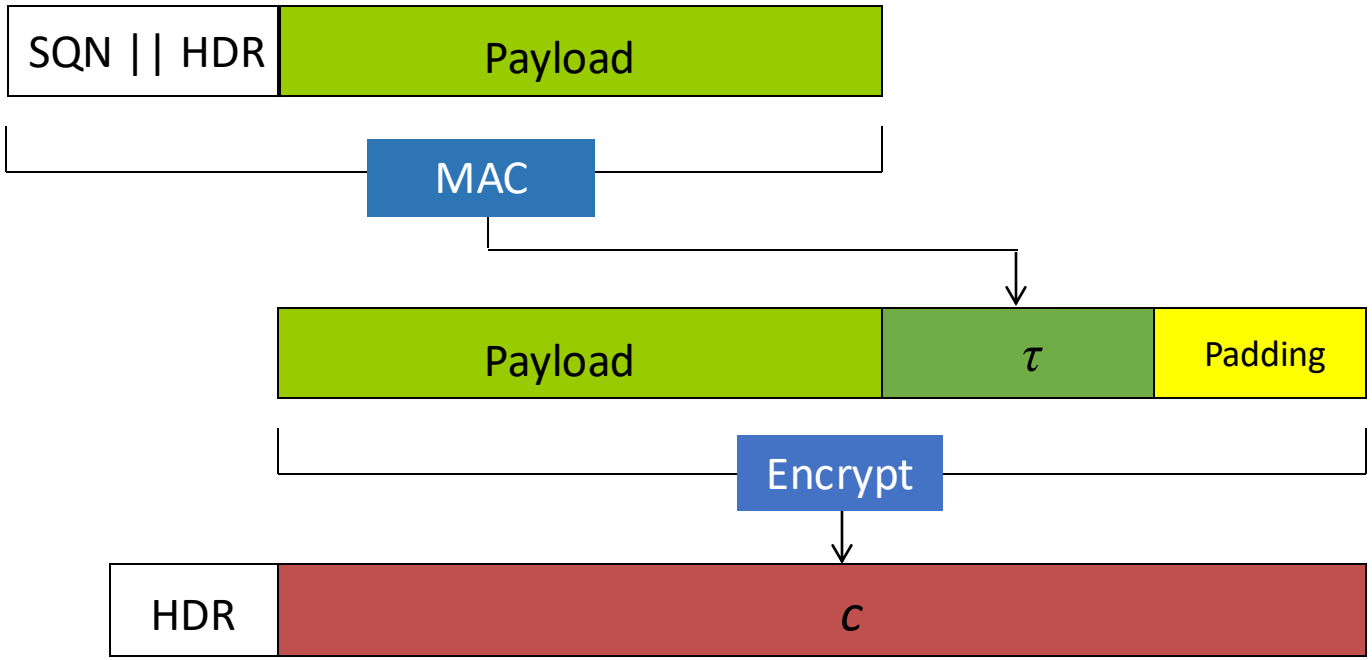
Example: SSH binary packet protocol (RFC 4253)



Stateful Encode-then-E&M (encrypt-and-MAC) construction:

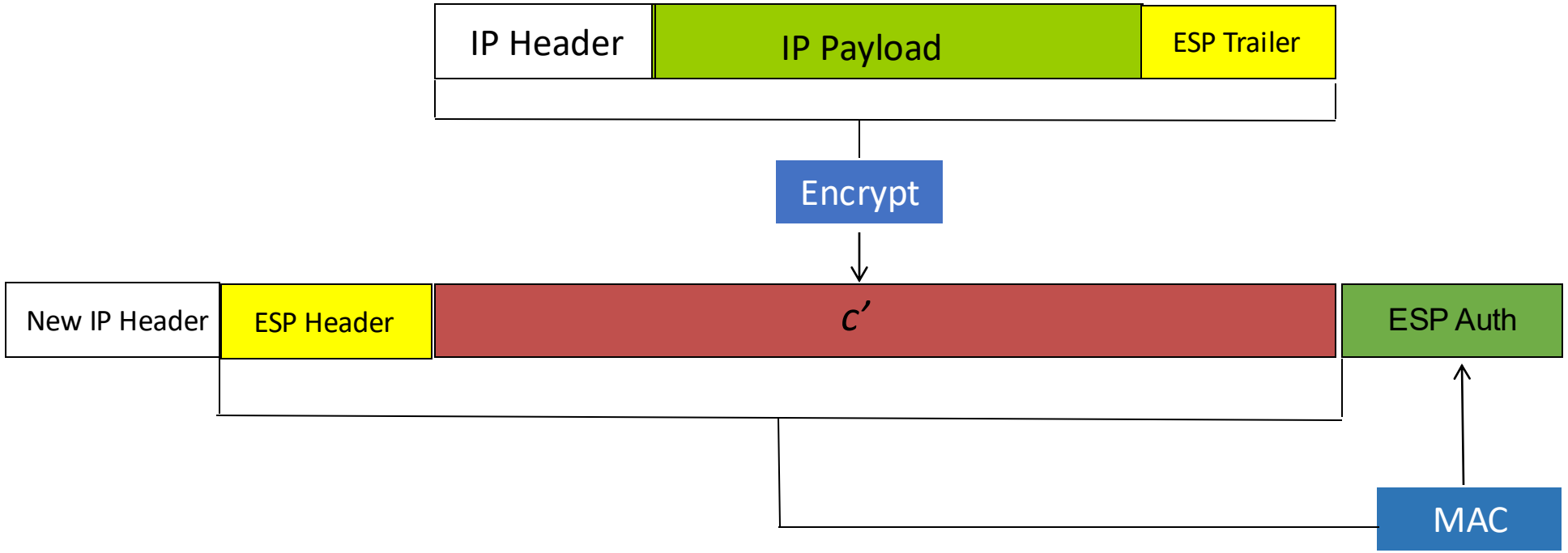
- packet length field measures the size of the packet: |PadLen|+ |Payload| + |Padding|
  - encrypted, so sequence of encrypted packets looks like a long string of random bytes
- Encryption options in RFC 4253: CBC mode; RC4
- AES-CTR defined in RFC 4344

Example: TLS record protocol: MAC-then-Encrypt



MAC	HMAC-MD5, HMAC-SHA1, HMAC-SHA256, …
Encrypt	CBC-AES128, CBC-AES256, CBC-3DES, RC4-128, …
Padding	“00” or “01 01” or “02 02 02” or …. or “FF FF….FF”

Example: IPsec ESP (RFC 4303): Encrypt-then-MAC



IP Encapsulating Security Payload (ESP) with enc and auth

Only EtM gives AE security in general!

EtM: compute $c' \leftarrow \mathrm{Enc}_{K_E}(m)$ and $\tau \leftarrow \mathrm{Tag}_{K_M}(c')$ and output $c = (c', \tau)$

- AE secure assuming encryption is IND-CPA secure and MAC is SUF-CMA secure

- Intuition:
  - MACing the ciphertext c' provides ciphertext integrity
  - IND-CPA security of encryption carries over to the composition

- Advantage: can check MAC on ciphertext, don't even decrypt if it fails
  - no temptation for programmer to "use the plaintext anyway" if MAC fails

Why E&M fails to be secure **in general**

Recall E&M:

- compute $c' \leftarrow \mathrm{Enc}_{KE}(m)$
- $\tau \leftarrow \mathrm{Tag}_{KM}(m)$
- output c = (c', τ)

- Suppose we have a SUF-CMA secure MAC scheme with tagging algorithm $\mathrm{Tag}_{KM}(m)$
- Define a new MAC scheme which outputs $\mathrm{Tag}_{KM}(m) \,||\, m_0$
- The new scheme is still SUF-CMA secure!
- What about the security of the resulting E&M scheme?

MtE can fail to be secure **in general**

Example

- Consider the following MtE encryption scheme
- MAC is provided by HMAC (SUF-CMA)
- Encryption scheme is provided by CBC-mode using simplified TLS padding (IND-CPA)

- KGen: select at random two keys: KM, KE
- Encryption: $c = \text{CBC-Enc}_{KE}(\text{TLS-PAD}(m \mid\mid \text{Tag}_{KM}(m)))$
- Decryption:
    1. Perform CBC-mode decryption
    2. Remove padding – possibility of padding error
    3. Perform MAC verification – possibility of MAC verification error

MtE can fail to be secure **in general**

- Decryption:
  1. Perform CBC-mode decryption
  2. Remove padding – possibility of padding error
  3. Perform MAC verification – possibility of MAC verification error
- If the errors at steps 2 and 3 are distinguishable, then we can carry out a padding oracle attack!
  - Padding error -> padding bad
  - MAC verification error -> padding good!
- This attack is a special case of a chosen-ciphertext attack (breaking IND-CCA)
- Thus, cannot be AE secure (AE security implies IND-CCA security)

Summary

- For an IND-CPA secure encryption scheme and a SUF-CMA secure MAC the MtE and E&M compositions not in general AE secure
- These compositions should be avoided wherever possible
- Specific instantiations of MtE (and E&M) may be AE secure
- The EtM composition can be proven to be AE secure for any IND-CPA secure encryption scheme and any SUF-CMA secure MAC

Dedicated AE constructions

Dedicated AE constructions

- So far, we have only seen *generic constructions* for AE schemes
  - Using IND-CPA encryption with SUF-CMA MAC composed by EtM

- Could we build dedicated Authenticated Encryption schemes, which could achieve AE security as one single primitive?

Motivation for dedicated AE constructions

- Combining off-the-shelf primitives
  - Can be prone to implementation errors
  - Requires two data passes (one for encryption, one for authentication)
  - Requires two keys (one for encryption, one for authentication)
- Some possible advantages of a dedicated AE design:
  - More secure and efficient constructions
  - Only requiring a single key
- This has been an active area of research in recent years
- We will look in detail at just two: **CCM** and **GCM**

CCM = Counter with CBC MAC

- An instantiation of **MtE** with M = CBC-MAC and E = CTR mode, using a 128-bit block cipher, e.g. AES

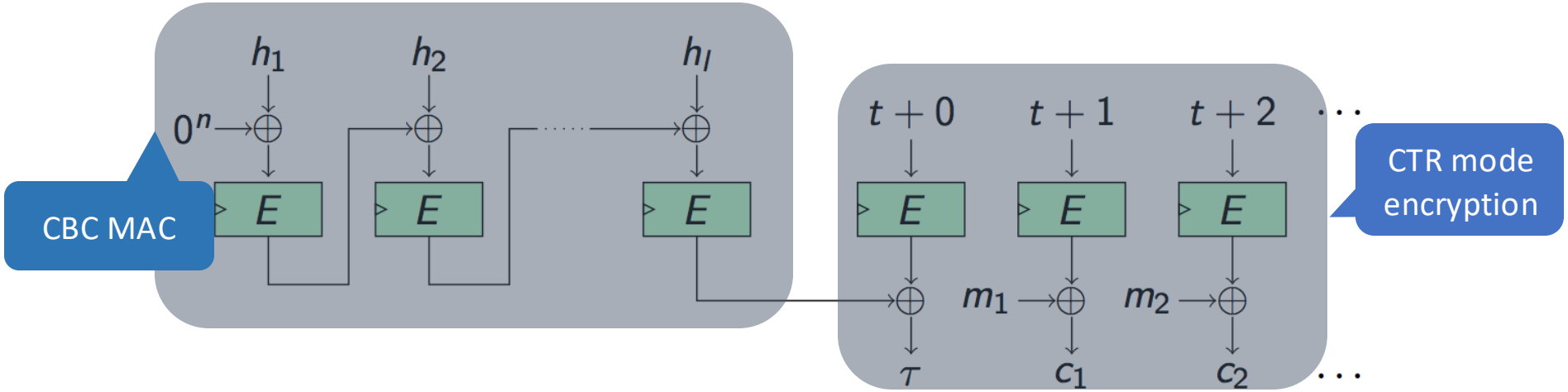
Tweaks:

- Use same key for "M" and "E" components (Bad idea in general! OK here)
- Apply CBC-MAC to the string:

$$h = N \,||\, \mathrm{len}(m)_{64} \,||\, m \,||\, \mathrm{len}(AD)_{64} \,||\, AD \,||\, \text{padding}$$

(where "||" means concatenate, $\mathrm{len}(X)_{64}$ means the 64-bit encoding of the length of string X)

- Initial counter value for CTR mode is $t = N \,||\, 0^{64}$, where N is the (64-bit) nonce



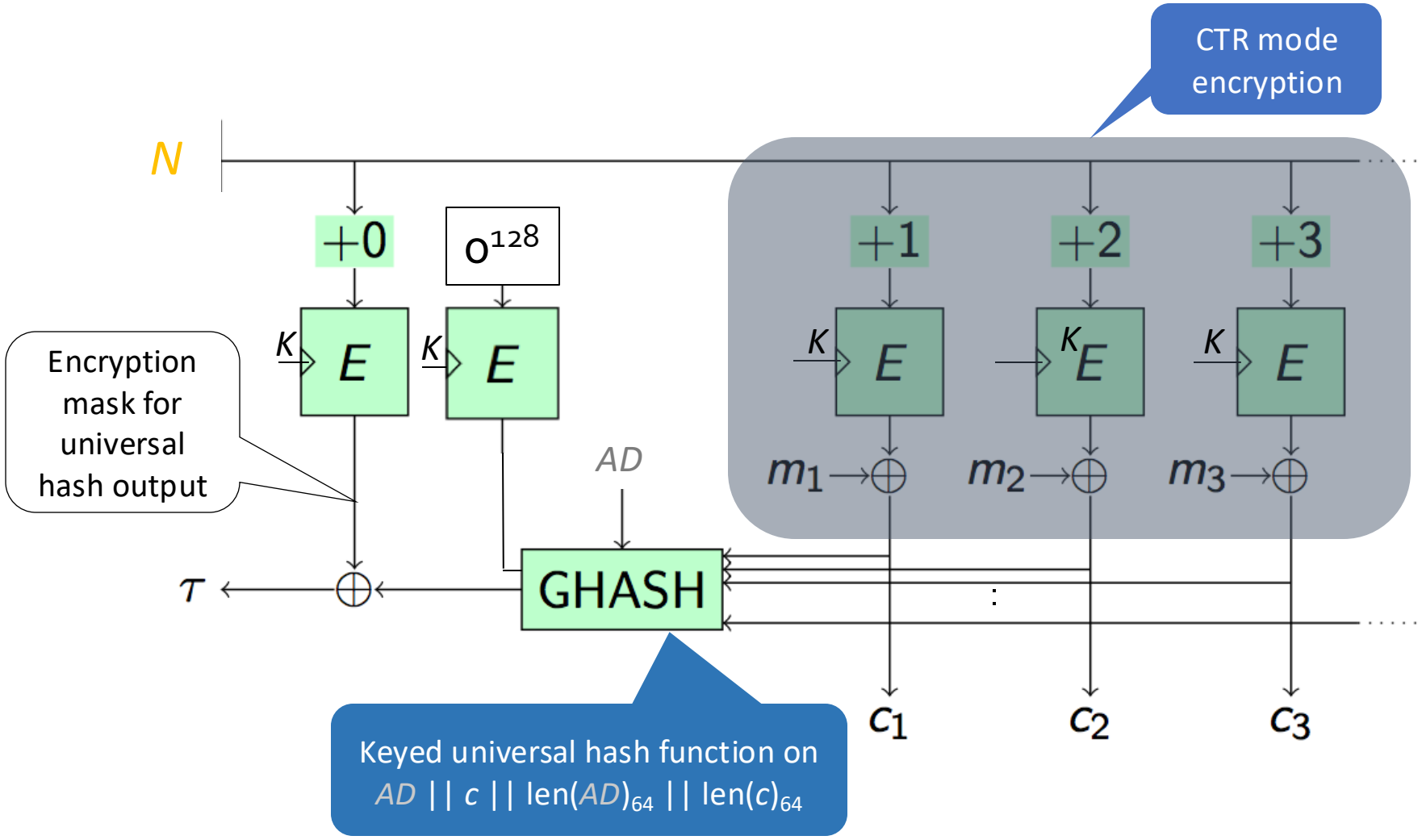
CCM features

- Quite slow: it needs 1 pass over associated data *AD* in CBC-MAC and two passes over the message *m,* one in CBC-MAC and one in CTR-mode encryption
- Only uses block cipher in “forward direction”, i.e. only “E” and no “D”
- Patent-free
- Used in WPA2, the successor to WEP and WPA/TKIP
- Standardised for use in IPsec and TLS 1.2
- Specified in full in RFC 3610 (https://tools.ietf.org/html/rfc3610)
- Has as a security proof based on block cipher being a pseudo-random permutation
- No known attacks (when implemented properly!)

GCM = Galois Counter Mode

- An instantiation of **EtM** with
  - E = CTR mode using a 128-bit block cipher, e.g. AES
  - M = a Wegman-Carter MAC
- Nonces *N* can be of arbitrary length
  - Special processing for 96-bit case for speed

GCM (for 96-bit nonces)



GCM features

- Faster than CCM:
  - Speed up comes from use of fast MAC algorithm built from universal hash function family called GHASH
- Only uses block cipher in "forward direction", i.e. only "E" and no "D"
- *AD* and *m* can be processed in block-wise fashion, no buffering required
- Has a security proof based on block cipher being a pseudo-random permutation
- No known attacks of significance (when implemented properly!)*
- **Security fails very badly if nonces are ever re-used with same key K**
- Patent-free
- Specified in full in NIST Special Publication SP800-38D (2007)
- Standardised for use in IPsec and TLS 1.2, and has become **widely used in TLS**

*Beware: some allowed parameter sets of GCM have "short" nonces (same for tags)

Further AE designs

- ChaCha20-Poly1305 is the default in recent versions of OpenSSH and available in TLS:
  - Similar to AES-GCM
  - Instead of AES-CTR it uses the ChaCha20 stream cipher
  - Instead of GHASH, it uses the Poly1305 MAC
- SHA-3 winner KECCAK can be adapted to produce an AE scheme Sponge construction for AE…

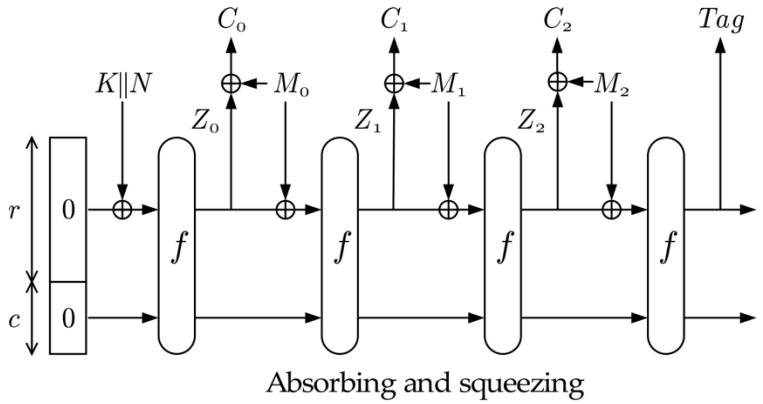


Figure source: J. Daemen's presentation at DIAC 2012 https://hyperelliptic.org/DIAC/

The CAESAR competition

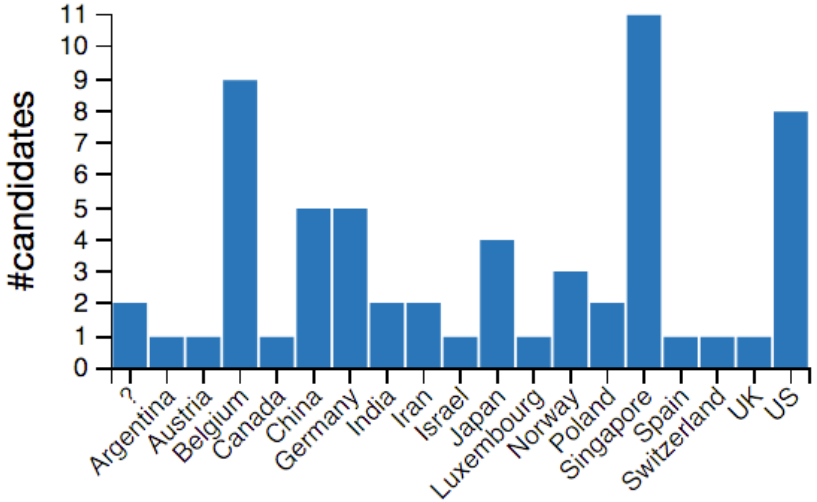
Competition for **A**uthenticated **E**ncryption: **S**ecurity, **A**pplicability, and **R**obustness (CAESAR)

- NIST-supported competition to identify a portfolio of authenticated ciphers that:
    1. Offer advantages over AES-GCM;
    2. Are suitable for widespread adoption

- Deadline for submissions March 2014
- Announcement of portfolio: Feb 2019
- All proposals were open for public evaluation
  http://competitions.cr.yp.to/caesar.html

CAESAR competition timeline

- Mar 2014: 57 submissions received
- Jul 2015: 30 second-round candidates announced
- Aug 2016 15 third-round candidates announced
- Mar 2018 Announcement of 7 finalists
- Feb 2019 Announcement of final portfolio (6 ciphers)



source: http://homes.esat.kuleuven.be/~eandreev/caesarviz/

CAESAR candidates features

- Type of construction:
  - Block Cipher-based
  - Stream cipher-based
  - Sponge-based
  - Permutation-based
- Various security levels (64-256 bits), availability of proof of security
- Implementation features: parallelizable, inverse-free, platform target
- Nonce-Misuse, RUP security

CAESAR final portfolio:

Ascon, ACORN, AEGIS-128, OCB, Deoxys-II, COLM

candidate	designers
Ascon, first choice for use case 1: home v1 v1.1 **v1.2**	Christoph Dobraunig, Maria Eichlseder, Florian Mendel, Martin Schläffer
ACORN, second choice for use case 1: v1 v2 **v3**	Hongjun Wu

candidate	designers
AEGIS-128 for use case 2: v1 **v1.1**	Hongjun Wu, Bart Preneel
OCB for use case 2: v1 **v1.1**	Ted Krovetz, Phillip Rogaway

candidate	designers
Deoxys-II, first choice for use case 3: home v1 ordering addendum v1.3 v1.4 **v1.41**	Jérémy Jean, Ivica Nikolić, Thomas Peyrin, Yannick Seurin
COLM, second choice for use case 3: v1pre **v1** addendum; superseding AES-COPA: v1 v2; and superseding ELmD: v1 clarification v2.0 v2.1	Elena Andreeva, Andrey Bogdanov, Nilanjan Datta, Atul Luykx, Bart Mennink, Mridul Nandi, Elmar Tischhauser, Kan Yasuda

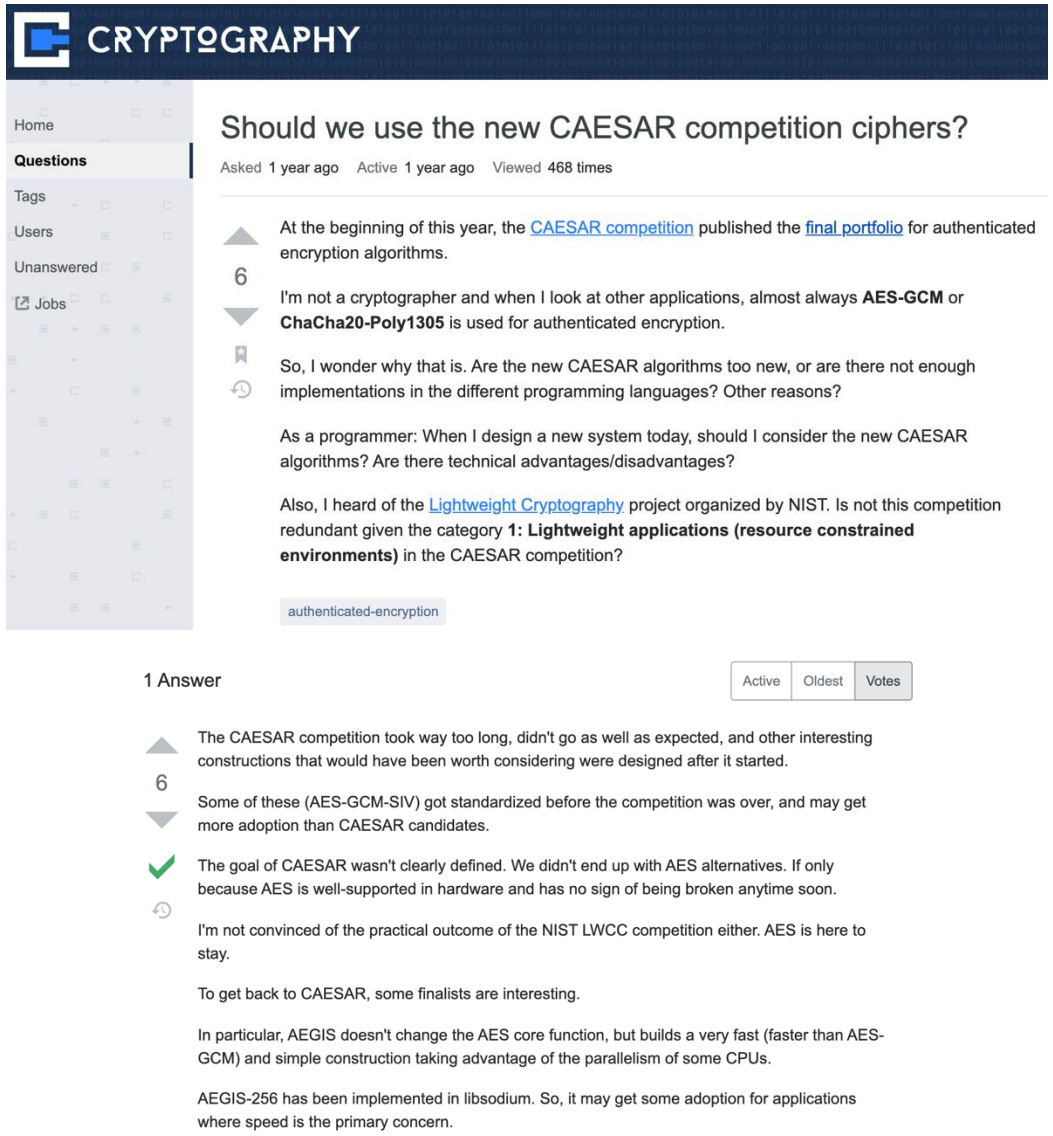
Use Case 1: Lightweight applications (resource constrained environments)
* critical: fits into small hardware area and/or small code for 8-bit CPUs
* desirable: natural ability to protect against side-channel attacks
* desirable: hardware performance, especially energy/bit
* desirable: speed on 8-bit CPUs
* message sizes: usually short (can be under 16 bytes), sometimes longer

Use Case 2: High-performance applications
* critical: efficiency on 64-bit CPUs (servers) and/or dedicated hardware
* desirable: efficiency on 32-bit CPUs (small smartphones)
* desirable: constant time when the message length is constant
* message sizes: usually long (more than 1024 bytes), sometimes shorter

Use Case 3: Defense in depth
* critical: authenticity despite nonce misuse
* desirable: limited privacy damage from nonce misuse
* desirable: authenticity despite release of unverified plaintexts
* desirable: limited privacy damage from release of unverified plaintexts
* desirable: robustness in more scenarios; e.g., huge amounts of data

Outcome

- In contrast to the NIST-run competitions (AES, SHA-3), there is (unfortunately) little evidence that CAESAR winning ciphers will become popular
- While the competition was onogoing, existing AE schemes (eg AES-GCM and ChaCha-Poly1305) and new ones (AES-GCM-SIV) were included in standards and became more widely used
- The NIST Lightweight Cryptography competition looked at standardising lightweight AEAD schemes: https://csrc.nist.gov/projects/lightweight-cryptography



Homework

- Read Chapter 8 of *Serious Cryptography*
- Learn more about AE, AES-GCM and ChaChaPoly-1305 in the Further Reading & Research section in Moodle
- Learn more about the CAESAR and NIST LWC competitions